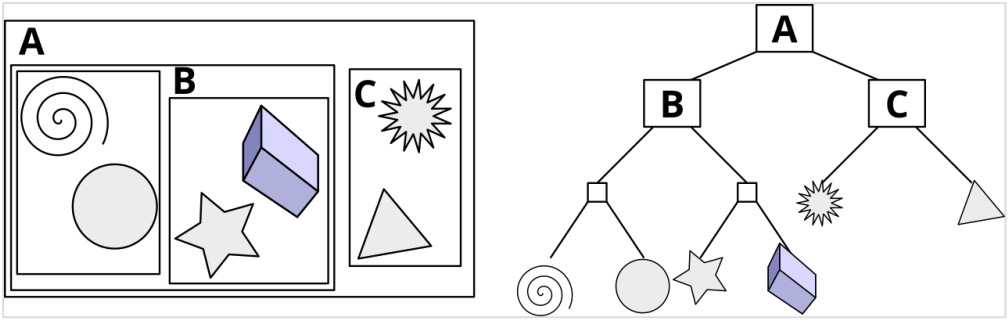
Bounding volume hierarchy

A **bounding volume hierarchy** (**BVH**) is a tree structure on a set of geometric objects. All geometric objects, which form the leaf nodes of the tree, are wrapped in bounding volumes. These nodes are then grouped as small sets and enclosed within larger bounding volumes. These, in turn, are also grouped and enclosed within other larger bounding volumes in a recursive fashion, eventually resulting in a tree structure with a single bounding volume at the top of the tree. Bounding volume hierarchies are used to support several operations on sets of geometric objects efficiently, such as in collision detection and ray tracing.



An example of a bounding volume hierarchy using rectangles as bounding volumes

Although wrapping objects in bounding volumes and performing collision tests on them before testing the object geometry itself simplifies the tests and can result in significant performance improvements, the same number of pairwise tests between bounding volumes are still being performed. By arranging the bounding volumes into a bounding volume hierarchy, the time complexity (the number of tests performed) can be reduced to logarithmic in the number of objects. With such a hierarchy in place, during collision testing, children volumes do not have to be examined if their parent volumes are not intersected (for example, if the bounding volumes of two bumper cars do not intersect, the bounding volumes of the bumpers themselves would not have to be checked for collision).

BVH design issues

The choice of bounding volume is determined by a trade-off between two objectives. On the one hand, bounding volumes that have a very simple shape need only a few bytes to store them, and intersection tests and distance computations are simple and fast. On the other hand, bounding volumes should fit the corresponding data objects very tightly. One of the most commonly used bounding volumes is an axis-aligned minimum bounding box. The axis-aligned minimum bounding box for a given set of data objects is easy to compute, needs only few bytes of storage, and robust intersection tests are easy to implement and extremely fast.

There are several desired properties for a BVH that should be taken into consideration when designing one for a specific application:[1]

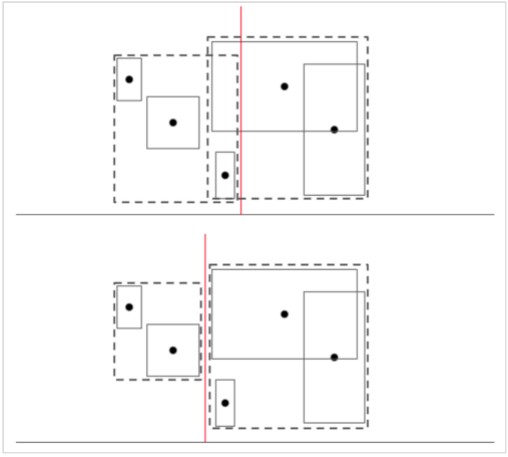
- The nodes contained in any given sub-tree should be near each other. The lower down the tree, the nearer the nodes should be to each other.
- Each node in the BVH should be of minimum volume.
- The sum of all bounding volumes should be minimal.

- Greater attention should be paid to nodes near the root of the BVH. Pruning a node near the root of the tree removes more objects from further consideration.
- The volume of overlap of sibling nodes should be minimal.
- The BVH should be balanced with respect to both its node structure and its content. Balancing allows as much of the BVH as possible to be pruned whenever a branch is not traversed into.

In terms of the structure of BVH, it has to be decided what degree (the number of children) and height to use in the tree representing the BVH. A tree of a low degree will be of greater height. That increases root-to-leaf traversal time. On the other hand, less work has to be expended at each visited node to check its children for overlap. The opposite holds for a high-degree tree: although the tree will be of smaller height, more work is spent at each node. In practice, binary trees (degree = 2) are by far the most common. One of the main reasons is that binary trees are easier to build.[2]

Construction

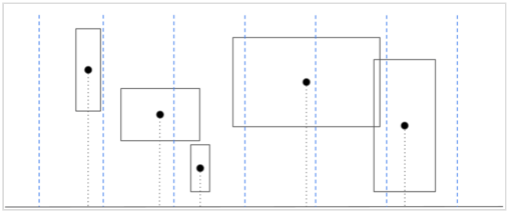
There are three primary categories of tree construction methods: top-down, bottom-up, and insertion methods.



BVH splitting plane selection that possibly lead to big overlap region (upper diagram) compared to a better selection (lower diagram)

Top-down methods

Top-down methods proceed by partitioning the input set into two (or more) subsets, bounding them in the chosen bounding volume, then continuing to partition (and bound) recursively until each subset consists of only a single object represented by a leaf node. Top-down methods are easy to implement, fast to construct and by far the most popular, but do not result in the best possible trees in general.



BVH construction with buckets of splitting planes for fast SAH criteria check

The most crucial part for this approach is to decide how to partition objects in a node's region among the node's children. This can be done with a splitting plane to split a node's bounding volume into partitions so that having to traverse many child nodes is minimized. Simply using the midpoint of volume centroids for splitting might be a sub-optimal choice, as illustrated in the figure, where a big overlap volume occurs. Hence, good splitting criteria such as the surface-area heuristic (SAH) are often used with equal-size buckets of splitting planes, so that only at these splitting points, re-calculation of SAH is required.

Bottom-up methods

Bottom-up methods start with the input set as the leaves of the tree and then group two (or more) of them to form a new (internal) node, proceed in the same manner until everything has been grouped under a single node (the root of the tree). Bottom-up methods are more difficult to implement, but likely to produce better trees in general. One study[3] indicates that in low-

dimensional space, the construction speed can be largely improved (to match or outperform top-down approaches) by sorting objects using a space-filling curve and applying approximate clustering based on this sequential order.

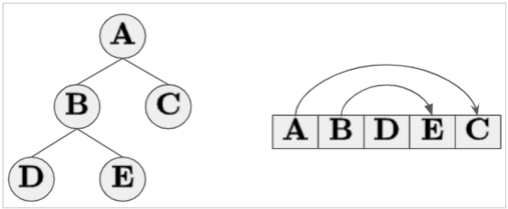
One example for this is the use of a Z-order curve (also known as Morton-order), where clusters can be found by simply taking a linear pass through a Morton-ordered array of leaves. Given the independent clusters of leaf nodes, sub-trees can be constructed in parallel and then further combined to form higher nodes. This parallelization makes the BVH construction very fast and can also be implemented in a hybrid manner, where all sub-trees are combined followed by a top-down approach.

Online methods

Both top-down and bottom-up methods are considered *off-line methods* as they both require all objects to be available before construction starts. *Insertion methods* build the tree by inserting one object at a time, starting from an empty tree. The insertion location should be chosen that causes the tree to grow as little as possible according to a cost metric. Insertion methods are considered *on-line methods* since they do not require all objects to be available before construction starts and thus allow updates to be performed at runtime.

Compact BVH for efficient traversal

After a BVH tree is constructed, it can then be converted to a compacted form for efficient traversal to improve overall system performance. The compact representation is often a linear array in memory, where the nodes of the BVH tree are stored in depth-first order. Hence, for a binary BVH tree, the first child of an internal node will be placed next to itself, and only the offset from the interior node to the index of the second child must be stored explicitly.



Visualization of an flattened BVH tree for efficient traversal [4]

The implementation of the traversal can be done without recursive function calls and only a stack is required to store the nodes to be visited next. The pseudocode provided below is a simple reference for ray tracing applications.

```
1  RayTracing(Ray, LinearBVHarr):
2      Let V be an empty stack
3      Put index 0 into V
4      while (V is not empty):
5          Let node B be the extracted top node in V
6          if (Ray intersects B's bound):
7              if (node is a leaf):
8                  Check all primitive objects in the bounded volumes
9                    and record the intersections
10             else:
11                 Put second child index of B on top of V
12                 Put first child index of B on top of V
```

Usage

Ray tracing

BVHs are often used in ray tracing to eliminate potential intersection candidates within a scene by omitting geometric objects located in bounding volumes which are not intersected by the current ray.[5] Additionally, as a common performance optimization, when only the closest intersection of the ray is of interest, while the ray tracing traversal algorithm is descending nodes, and multiple child nodes intersect the ray, the traversal algorithm will consider the closer volume first, and if it finds an intersection there, which is definitively closer than any possible intersection in a second (or other) volume (i.e., volumes are non-overlapping), it can safely ignore the second volume. This only requires a small change in line 11-12 of the above pseudo-code. Similar optimizations during BVH traversal can be employed when descending into child volumes of the second volume, to restrict further search space and thus reduce traversal time.

Additionally, many specialized methods were developed for BVHs, especially ones based on AABB (axis-aligned bounding boxes), such as parallel building, SIMD accelerated traversal, good split heuristics (SAH - surface-area heuristic is often used in ray tracing), wide trees (4-ary and 16-ary trees provide some performance benefits, both in build and query performance for practical scenes), and quick structure update (in real time applications objects might be moving or deforming spatially relatively slowly or be still, and same BVH can be updated to be still valid without doing a full rebuild from scratch).

Scene-graph management

BVHs also naturally support inserting and removing objects without full rebuild, but with resulting BVH having usually worse query performance compared to full rebuild. To solve these problems (as well as quick structure update being sub-optimal), the new BVH could be built asynchronously in parallel or synchronously, after sufficient change is detected (leaf overlap is big, number of insertions and removals crossed the threshold, and other more refined heuristics).

BVHs can also be combined with scene graph methods, and geometry instancing, to reduce memory usage, improve structure update and full rebuild performance, as well as guide better object or primitive splitting.

Collision detection

BVHs are often used for accelerating collision detection computation. In the context of cloth simulation, BVHs are used to compute collision between a cloth and itself as well as with other objects.[6]

Distance calculation between set of objects

Another powerful use case for BVH is pair-wise distance computation. A naive approach to find the minimum distance between two set of objects would compute the distance between all of the pair-wise combinations. A BVH allows us to efficiently prune many of the comparisons without needing

to compute potentially elaborate distance between the all objects. Pseudo code for computing pairwise distance between two set of objects and approaches for building BVH, well suited for distance calculation is discussed here [4]

Hardware-enabled BVH acceleration

Accelerating ray tracing

BVH can significantly accelerate ray tracing applications by reducing the number of ray-surface intersection calculations. Hardware implementation of BVH operations such as traversal can further accelerate ray-tracing. Currently, real-time ray tracing is available on multiple platforms. Hardware implementation of BVH is one of the key innovations making it possible.

Nvidia RT Cores

In 2018, Nvidia introduced RT Cores with their Turing GPU architecture as part of the RTX platform. RT Cores are specialized hardware units designed to accelerate BVH traversal and ray-triangle intersection tests.[7] The combination of these key features enables real-time ray tracing that can be use for video games.[8] as well as design applications.

AMD RDNA 2/3

AMD's RDNA (Radeon DNA) architecture, introduced in 2019, has incorporated hardware-accelerated ray tracing since its second iteration, RDNA 2. The architecture uses dedicated hardware units called Ray Accelerators to perform ray-box and ray-triangle intersection tests, which are crucial for traversing Bounding Volume Hierarchies (BVH).[9] In RDNA 2 and 3, the shader is responsible for traversing the BVH, while the Ray Accelerators handle intersection tests for box and triangle nodes.[10]

Usage of HW enabled BVH beyond ray tracing

Originally designed to accelerate ray tracing, researchers are now exploring ways to leverage fast BVH traversal to speed up other applications. These include determining the containing tetrahedron for a point,[11] enhancing granular matter simulations,[12] and performing nearest neighbor calculations.[13] Some methods repurpose Nvidia's RT core components by reframing these tasks as ray-tracing problems.[12] This direction seems promising as substantial speedups in performance are reported across the various applications.

See also

- Binary space partitioning, octree, *k*-d tree
- R-tree, R+-tree, R*-tree and X-tree
- M-tree
- Sweep and prune
- Hierarchical clustering
- OptiX

References

1. Ericson, Christer (2005). "§6.1 Hierarchy Design Issues" (https://books.google.com/books?id=WGpL6Sk9qNAC&pg=PA236). *Real-Time collision detection*. Morgan Kaufmann Series in Interactive 3-D Technology. Morgan Kaufmann. pp. 236–7. ISBN 1-55860-732-3.
2. Ericson 2005, p. 238
3. Gu, Yan; He, Yong; Fatahalian, Kayvon; Blelloch, Guy (2013). "Efficient BVH Construction via Approximate Agglomerative Clustering" (https://cs.cmu.edu/~blelloch/papers/GHFB13.pdf) (PDF). *HPG '13: Proceedings of the 5th High-Performance Graphics Conference*. ACM. pp. 81–88. CiteSeerX 10.1.1.991.3441 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.991.3441). doi:10.1145/2492045.2492054 (https://doi.org/10.1145%2F2492045.2492054). ISBN 978-1-4503-2135-8. S2CID 2585433 (https://api.semanticscholar.org/CorpusID:2585433).
4. Ytterlid, Robin; Shellshear, Evan (January 27, 2015). "BVH Split Strategies for Fast Distance Queries" (http://jcgt.org/published/0004/01/01/). *Journal of Computer Graphics Techniques*. **4** (1): 1–25. ISSN 2331-7418 (https://search.worldcat.org/issn/2331-7418). Retrieved October 13, 2024.
5. Günther, J.; Popov, S.; Seidel, H.-P.; Slusallek, P. (2007). "Realtime Ray Tracing on GPU with BVH-based Packet Traversal". *2007 IEEE Symposium on Interactive Ray Tracing*. IEEE. pp. 113–8. CiteSeerX 10.1.1.137.6692 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.137.6692). doi:10.1109/RT.2007.4342598 (https://doi.org/10.1109%2FRT.2007.4342598). ISBN 978-1-4244-1629-5. S2CID 2840180 (https://api.semanticscholar.org/CorpusID:2840180).
6. Bridson, Robert; Fedkiw, Ronald; Anderson, John (25 June 1997). "Robust treatment of collisions, contact and friction for cloth animation". *ACM Trans. Graph*. **21** (3). Berlin: Association for Computing Machinery: 594–603. doi:10.1145/566654.566623 (https://doi.org/10.1145%2F566654.566623).
7. "NVIDIA Turing GPU Architecture" (https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf) (PDF). *NVIDIA*. Retrieved 2024-10-20.
8. "The NVIDIA Turing GPU Architecture Deep Dive: Prelude to GeForce RTX" (https://web.archive.org/web/20180914174557/https://www.anandtech.com/show/13282/nvidia-turing-architecture-deep-dive). *AnandTech*. Archived from the original (https://www.anandtech.com/show/13282/nvidia-turing-architecture-deep-dive) on September 14, 2018. Retrieved 2024-10-20.
9. "RDNA 2 hardware raytracing" (https://interplayoflight.wordpress.com/2020/12/27/rdna-2-hardware-raytracing/). *Interplay of Light*. 27 December 2020. Retrieved 2024-10-20.
10. "Raytracing on AMD's RDNA 2/3, and Nvidia's Turing and Pascal" (https://chipsandcheese.com/p/raytracing-on-amds-rdna-2-3-and-nvidias-turing-and-pascal). *Chips and Cheese*. 2023-03-22. Retrieved 2024-10-20.
11. Wald, Ingo; Usher, Will; Morrical, Nathan; Lediaev, Laura; Pascucci, Valerio (2019). "RTX Beyond Ray Tracing: Exploring the Use of Hardware Ray Tracing Cores for Tet-Mesh Point Location" (https://diglib.eg.org/handle/10.2312/hpg20191189). *High-Performance Graphics - Short Papers*: 7 pages. doi:10.2312/HPG.20191189 (https://doi.org/10.2312%2FHPG.20191189). ISBN 978-3-03868-092-5. ISSN 2079-8687 (https://search.worldcat.org/issn/2079-8687).
12. Zhao, Shiwei; Zhao, Jidong (November 1, 2023). "Revolutionizing granular matter simulations by high-performance ray tracing discrete element method for arbitrarily-shaped particles" (https://linkinghub.elsevier.com/retrieve/pii/S0045782523004942). *Computer Methods in Applied Mechanics and Engineering*. **416** 116370. doi:10.1016/j.cma.2023.116370 (https://doi.org/10.1016%2Fj.cma.2023.116370).

13. Zhu, Yuhao (2022-04-02). "RTNN: Accelerating neighbor search using hardware ray tracing" (https://dl.acm.org/doi/10.1145/3503221.3508409). *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. ACM. pp. 76–89. doi:10.1145/3503221.3508409 (https://doi.org/10.1145%2F3503221.3508409). ISBN 978-1-4503-9204-4.

External links

- BVH in JavaScript (https://github.com/imbcmdth/jsBVH).
- Dynamic BVH in C# (https://www.codeproject.com/Articles/832957/Dynamic-Bounding-Volume-Hiearchy-in-Csharp)
- Intel Embree open source BVH library (https://www.embree.org/)
- How to build a BVH - Part 1 (https://jacco.ompf2.com/2022/04/13/How-to-build-a-bvh-part-1-basics/)
- AABB Trees (https://doc.cgal.org/latest/Manual/packages.html#PkgAABBTree) in CGAL, the Computational Geometry Algorithms Library